Individual Contribution Report

Operating Systems Final Project · MiniOS Ekoms

Student Name: CIUPAN
Group: Group 8 - SC1063 (MiniOS Ekoms)
Assigned Role: User/System Developer

What did I contribute to the project?

I developed the user authentication and system information modules (users.py and sysinfo.py) for MiniOS Ekoms. I worked closely with the Project Leader to integrate these modules with main.py, enabling key shell commands like whoami, users, logout, systeminfo, and version. Overall, my work controls user access, session security, and system identity display.

What features did I personally develop?

- **User Accounts:** Created built-in admin and student roles.
- **Secure Passwords:** Used SHA-256 hashing to hide and compare passwords safely.
- **Session Control:** Built login, logout, and active user tracking functions.
- **Masked Input:** Showed * characters while typing passwords in Windows CMD.
- **Login Screen:** Added a 3-attempt login limit that locks out unauthorized users.
- **System Specs Display:** Formatted system info (OS name, version, CPU, memory) in a central place.
- **Command Support:** Handled the underlying logic for all user and system commands.

What problems did I encounter?

Windows CMD hid typed passwords completely instead of displaying asterisks. Storing plain-text passwords posed a major security risk for the system. Processing special keys like Backspace, Enter, and Ctrl+C manually proved difficult. Windows-specific code caused crashes when run on Mac and Linux systems. Allowing unlimited login attempts was unsafe, and logging out failed to reset the screen cleanly. Centralized system details were inconsistent across different parts of the program.

How did I solve those problems?

I wrote a custom function that reads keystrokes one by one and displays asterisks. All passwords were hashed with SHA-256 before saving or checking them. Manual key checks for Enter, Backspace, and Ctrl+C were added directly inside the input loop. A try/except fallback was implemented so non-Windows systems safely use the getpass module instead. Logins were limited to three attempts, and logout was designed to cleanly return to the login screen. Finally, the OS name and version were defined in a single shared file so every module remains consistent.

What did I learn from this project?
Modular design lets separate operating system components communicate easily through clear functions. Password hashing protects users by converting raw passwords into unreadable codes. Cross-platform code uses fallbacks to work smoothly across different operating systems. Reading terminal input key by key allows precise handling of keys like Backspace and Enter. Clear error messages keep command-line interfaces helpful and easy to use. Team collaboration relies on Git, shared code agreements, and combined testing to merge work smoothly.

Evidence of my contribution:

GitHub repository: https://github.com/OtitsDub/Group8-SC1063
Source code: Main.py
Documentation: README.md

Screenshots:



